

Dateien und Verzeichnisse

Dateien

- .NET hat fertige Klassen zum erstellen, lesen, schreiben und löschen von Dateien
- StreamWriter und StreamReader sind die Standardklassen für den Dateizugriff
- können unterschiedliche Kodierungen abbilden
- Funktionen sind ähnlich wie bei der Konsole
- Streams müssen geöffnet und auch geschlossen werden
 - GC greift hier nicht ein, da geöffnete Dateien als „unmanaged ressource“ gelten

StreamWriter – System.IO

- mit einem StreamWriter können Dateien erstellt und Inhalt in diese geschrieben werden
- über die Eigenschaft Encoding kann die Kodierung der Ausgabe festgelegt werden
- mit Write() und WriteLine() können Inhalte an den Stream übergeben werden

```
StreamWriter sw = new StreamWriter("Testdatei.txt");  
sw.WriteLine("Testzeile1");  
sw.WriteLine("Testzeile2");  
sw.Close();
```

StreamReader – System.IO

- mit einem StreamReader können Inhalte einer Datei ausgelesen werden
- über die Eigenschaft EndOfStream kann geprüft werden ob der Stream noch weitere Daten enthält
- mit Read() und ReadLine() können Inhalte aus dem Stream gelesen werden

```
StreamReader sr = new StreamReader("Testdatei.txt");  
while (!sr.EndOfStream)  
{  
    string zeile = sr.ReadLine();  
}  
sr.Close();
```

Using-Statement

- Klassen, welche externe Ressourcen wie Dateien oder Datenbankserver verwalten, sollten das IDisposable-Interface implementieren
- Dispose() räumt alle externen Ressourcen auf, d.h. Dateien schließen oder DB-Verbindung lösen
- Mittels dem using-Statement wird Dispose() automatisch aufgerufen sobald das Ende des Using-Blockes erreicht wurde

```
using(StreamReader reader = new StreamReader("test.txt"))
{
    //Lese-Operationen
    //Am Ende des Blockes wird automatisch reader.Dispose() aufgerufen
}
```

SaveFileDialog – System.Windows.Forms

- mit einem SaveFileDialog kann ein Pfad und ein Dateiname ausgewählt werden
- die Eigenschaft Filter ermöglicht es nur bestimmte Dateiendungen zuzulassen
- über die Eigenschaft FileName kann man eine Pfadangabe vordefinieren bzw. den gewählten Pfad abrufen
- In WPF: using Microsoft.Win32

```
SaveFileDialog saveFileDialog = new SaveFileDialog();
saveFileDialog.FileName = "Personen.pl";
saveFileDialog.Filter = "Personenliste|*.pl|Textdokument|*.txt";
if (saveFileDialog.ShowDialog() == DialogResult.OK)
{
    string pfadZuDatei = saveFileDialog.FileName;
}
```

OpenFileDialog – System.Windows.Forms

- gleiche Funktionalität wie SaveFileDialog
- der Dateipfad muss existieren um den Dialog zu bestätigen
- In WPF: using Microsoft.Win32

```
OpenFileDialog openFileDialog = new OpenFileDialog();
openFileDialog.FileName = "Personen.pl";
openFileDialog.Filter = "Personenliste|*.pl|Textdokument|*.txt";
if (openFileDialog.ShowDialog() == DialogResult.OK)
{
    string pfadZuDatei = openFileDialog.FileName;
}
```

JSON – Newtonsoft.Json (NuGet-Paket)

- ein Objekt kann mit dem aktuellen Zustand bspw. in eine Datei geschrieben und später wieder eingelesen werden
- die Klasse JsonConvert kümmert sich um die Konvertierung der Inhalte
- mit der Methode SerializeObject der Klasse JsonConvert kann ein beliebiges Objekt in einen JSON-String konvertiert werden (serialisiert)
- mit der Methode DeserializeObject mit der Typangabe <T> der Klasse JsonConvert kann ein beliebiges Objekt in einen JSON-String konvertiert werden (serialisiert)

```
List<string> strings = new List<string> { "Wert1", "Wert2", "Wert3" };
```

```
string jsonString = JsonConvert.SerializeObject(strings);
```

```
List<string> stringListe = JsonConvert.DeserializeObject<List<string>>(jsonString);
```

JSON – System.Text.Json (ab .NET Core 3.0)

- neuer JSON Konverter direkt in .NET Core enthalten
- weniger Funktion als Newtonsoft.Json
- eine referenzierte Schleife führt zu einer Fehlermeldung
- um einiges schneller als Newtonsoft.Json
- ist jedoch Typabhängig (int => int only)
Case sensitiv, Eigenschaftsnamen nur mit doppeltem Anführungszeichen

```
List<string> strings = new List<string> { "Wert1", "Wert2", "Wert3" };
```

```
string jsonString = JsonSerializer.Serialize(strings);
```

```
List<string> stringListe = JsonSerializer.Deserialize<List<string>>(jsonString);
```

XML – System.Xml.Serialization

- mit einem XmlSerializer kann ein Objekt mithilfe eines TextWriters (bspw. StreamWriter) direkt in eine Datei geschrieben werden
- im Unterschied zu den JSON-Klassen muss bei XML ein eigenes Objekt der Klasse XmlSerializer erzeugt werden, welches im Konstruktor eine Instanz des Typs des zu serialisierenden Objektes haben muss
- über die Methode Serialize des XmlSerializer Objektes wird der TextWriter sowie das zu serialisierende Objekt übergeben

```
List<string> liste = new List<string> { "Wert1", "Wert2", "Wert3" };  
  
XmlSerializer serializer = new XmlSerializer(typeof(List<string>));  
  
StreamWriter sw = new StreamWriter("liste.xml");  
serializer.Serialize(sw, liste);  
sw.Close();
```